

4.1 From a Grammar to an LL(1) Parser

1. Eliminating left recursion

The grammar $S \rightarrow \text{id}(L) \mid S; \text{id}(L), L \rightarrow \varepsilon \mid M, M \rightarrow \text{id} \mid M, \text{id}$ has immediate left recursion in S and M . Introducing helpers S' and M' yields the equivalent right-recursive grammar: $S \rightarrow \text{id}(L)S', S' \rightarrow ; \text{id}(L)S' \mid \varepsilon, L \rightarrow M \mid \varepsilon, M \rightarrow \text{id} M', M' \rightarrow , \text{id} M' \mid \varepsilon$. To parse the whole input, extend it with $S_0 \rightarrow S\#$.

2. FIRST_1 and FOLLOW_1

FIRST sets:

- $\text{FIRST}(S_0) = \{\text{id}\}$ (through S)
- $\text{FIRST}(S) = \{\text{id}\}$
- $\text{FIRST}(S') = \{;, \varepsilon\}$
- $\text{FIRST}(L) = \{\text{id}, \varepsilon\}$
- $\text{FIRST}(M) = \{\text{id}\}$
- $\text{FIRST}(M') = \{,, \varepsilon\}$

FOLLOW sets (using S_0 as start symbol):

- $\text{FOLLOW}(S_0) = \{\#\}$
- $\text{FOLLOW}(S) = \{\#\}$
- $\text{FOLLOW}(S') = \{\#\}$ (end of S)
- $\text{FOLLOW}(L) = \{)\}$ (always followed by a right parenthesis)
- $\text{FOLLOW}(M) = \{)\}$ (because $L \rightarrow M$)
- $\text{FOLLOW}(M') = \{)\}$

3. LL(1) parse table (rows = non-terminals, columns = lookahead)

	id	(	)	;	,	#
S ₀	S #					
S	id(L)S'					
S'			;id(L)S'			ε
L	M		ε			
M	id M'					
M'			ε		,idM'	

No entry has two productions, so the grammar is LL(1).

4. Successful LL(1) run for $\text{id}(); \text{id}(\text{id}, \text{id})\#$

Stack is written bottom-to-top from left to right; the top of stack is on the right.

step	stack	input	action
0	S0	id () ; id (id , id) #	start
1	# S	id () ; id (id , id) #	expand S0 -> S #
2	# S') L (id	id () ; id (id , id) #	expand S -> id(L)S'
3	# S') L (	() ; id (id , id) #	match id
4	# S') L	) ; id (id , id) #	match (
5	# S')	) ; id (id , id) #	L -> ϵ (lookahead ')')
6	# S'	; id (id , id) #	match)
7	# S') L (id ;	; id (id , id) #	expand S' -> ;id(L)S'
8	# S') L (id	id (id , id) #	match ;
9	# S') L (	(id , id) #	match id
10	# S') L	id , id) #	match (
11	# S') M	id , id) #	L -> M
12	# S') M' id	id , id) #	expand M -> id M'
13	# S') M'	, id) #	match id
14	# S') M' id ,	, id) #	M' -> ,idM'
15	# S') M' id	id) #	match ,
16	# S') M'	) #	match id
17	# S')	) #	M' -> ϵ
18	# S'	#	match)
19	#	#	S' -> ϵ
20			match # / accept

The input is fully consumed and the stack empties after the end marker, so the run succeeds.

4.2 Item Pushdown Automaton

Grammar with synthetic start: $S' \rightarrow S, S \rightarrow aB, A \rightarrow B \mid \epsilon, B \rightarrow bBa \mid aA$. Input word: "abaa". Items use a dot to mark the current position.

step	stack (bottom $\rightarrow$ top)	remaining input	action
0	[S' $\rightarrow$. S]	abaa	expand start

1	[S' → . S] [S → . a B]	abaa	expand S
2	[S' → . S] [S → a . B]	baa	shift 'a'
3	[S' → . S] [S → a . B] [B → . b B a]	baa	expand B with 'b' branch
4	[S' → . S] [S → a . B] [B → b . B a]	aa	shift 'b'
5	[S' → . S] [S → a . B] [B → b . B a] [B → . a A]	aa	expand inner B with 'a'
6	[S' → . S] [S → a . B] [B → b . B a] [B → a . A]	a	shift 'a'
7	[S' → . S] [S → a . B] [B → b . B a] [B → a . A] [A → .]	a	expand A -> ε
8	[S' → . S] [S → a . B] [B → b . B a] [B → a A .]	a	reduce A
9	[S' → . S] [S → a . B] [B → b B . a]	a	reduce inner B
10	[S' → . S] [S → a . B] [B → b B a .]		shift 'a'
11	[S' → . S] [S → a B .]		reduce outer B
12	[S' → S .]		reduce S and accept

The final stack [S' → S .] proves the item PDA accepts the input abaa.